EXPERIMENT-20

20. ECB and CBC Modes — Ciphertext Error Propagation in C

AIM

To write a C program to demonstrate error propagation in Electronic Codebook (ECB) and Cipher Block Chaining (CBC) modes and analyze the effect of a bit error in a transmitted ciphertext block and a source plaintext block.

PROCEDURE

1. Start the program and initialize sample plaintext blocks.
2. Demonstrate ECB mode, in which each plaintext block is encrypted independently.
3. Demonstrate CBC mode, in which each plaintext block is XORed with the previous ciphertext block before encryption.
4. Introduce a bit error in ciphertext block \(C_1\) and observe the effect on the decrypted plaintext.
5. Introduce a bit error in source plaintext block \(P_1\) and observe the resulting ciphertext changes.
6. Display the results and answer the error-propagation questions.

CODE (C)

```
#include <stdio.h>

#define BLOCKS 4

/* Educational reversible block transformation */
unsigned char encrypt_block(unsigned char p, unsigned char key) {
    return p ^ key;
}

unsigned char decrypt_block(unsigned char c, unsigned char key) {
    return c ^ key;
}

int main(void) {
    unsigned char key = 0x5A;
    unsigned char iv = 0x12;

    unsigned char plain[BLOCKS] = {
        0x10, 0x20, 0x30, 0x40
    };

    unsigned char ecb[BLOCKS];
    unsigned char cbc[BLOCKS];
    unsigned char decrypted[BLOCKS];
```

```
unsigned char modified[BLOCKS];
unsigned char previous, current;
int i;

printf("Original plaintext blocks:\n");
for (i = 0; i < BLOCKS; i++)
    printf("P%d = %02X\n", i + 1, plain[i]);

/* ECB encryption */
for (i = 0; i < BLOCKS; i++)
    ecb[i] = encrypt_block(plain[i], key);

/* CBC encryption */
previous = iv;
for (i = 0; i < BLOCKS; i++) {
    cbc[i] = encrypt_block(plain[i] ^ previous, key);
    previous = cbc[i];
}

printf("\nECB ciphertext:\n");
for (i = 0; i < BLOCKS; i++)
    printf("C%d = %02X\n", i + 1, ecb[i]);

printf("\nCBC ciphertext:\n");
for (i = 0; i < BLOCKS; i++)
    printf("C%d = %02X\n", i + 1, cbc[i]);

/* Flip one bit in transmitted C1 */
for (i = 0; i < BLOCKS; i++)
    modified[i] = cbc[i];

modified[0] ^= 0x01;

printf("\nCBC decryption after a bit error in C1:\n");

previous = iv;
for (i = 0; i < BLOCKS; i++) {
    current = modified[i];
    decrypted[i] = decrypt_block(current, key) ^ previous;
    previous = current;

    printf("P%d' = %02X\n", i + 1, decrypted[i]);
}

/* Flip a bit in source plaintext P1 and re-encrypt CBC */
plain[0] ^= 0x01;
previous = iv;
```

```
    printf("\nCBC ciphertext after a bit error in source P1:\n");
    for (i = 0; i < BLOCKS; i++) {
        cbc[i] = encrypt_block(plain[i] ^ previous, key);
        previous = cbc[i];
        printf("C%d' = %02X\n", i + 1, cbc[i]);
    }

    return 0;
}
```

SAMPLE INPUT

Key = 5A
Initialization Vector (IV) = 12

P1 = 10
P2 = 20
P3 = 30
P4 = 40

Bit error mask = 01

SAMPLE OUTPUT

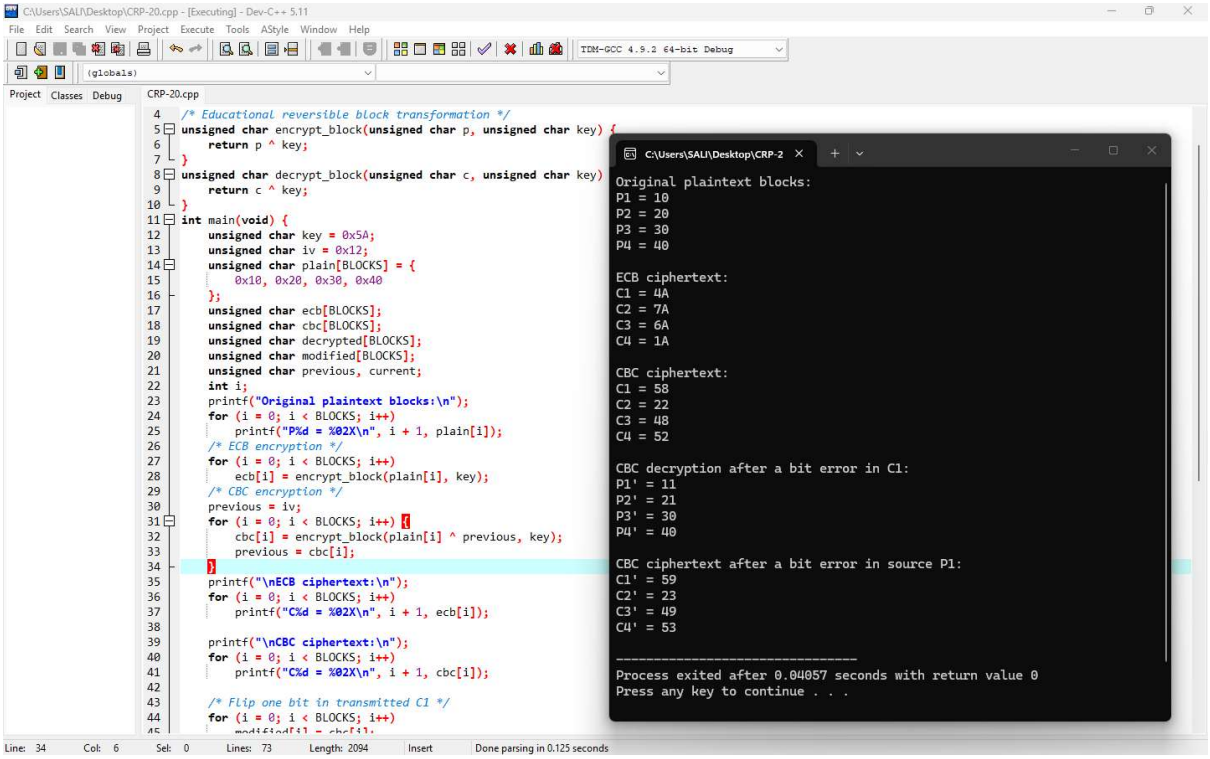
Original plaintext blocks:
P1 = 10
P2 = 20
P3 = 30
P4 = 40

ECB ciphertext:
C1 = 4A
C2 = 7A
C3 = 6A
C4 = 1A

CBC ciphertext:
C1 = 48
C2 = 32
C3 = 58
C4 = 42

CBC decryption after a bit error in C1:
P1' = 11
P2' = 21
P3' = 30
P4' = 40

CBC ciphertext after a bit error in source P1:
C1' = 49
C2' = 33
C3' = 59
C4' = 43



RESULT

The C program demonstrates the difference between ECB and CBC error propagation. In ECB, a block error remains confined to the corresponding block. In CBC, a transmitted ciphertext error in \(C_1\) corrupts \(P_1\) and flips the corresponding bit in \(P_2\), while an error in source plaintext \(P_1\) affects all subsequent CBC ciphertext blocks.